

WIE

Message System

Chat • Media • Block • Report • Permissions
Complete API Documentation for React Native Developers
Chat Service: port 5004 • All routes require Bearer token • v1.0

Table of Contents

PART A — Core Chat API (/api/wie-chat)

1. Service Overview & Authentication
2. Chat Permissions Model
3. Create or Get Chat (POST /create)
4. List My Chats (GET /list)
5. Send Text Message (POST /send)
6. Get Chat Messages (GET /:chatId/messages)
7. Chat Suggestions & User Search
8. Message Requests (GET /requests, POST /:chatId/accept, POST /:chatId/decline)
9. Mark Read / Unread
10. Unread Counts
11. Delete Message, Delete Chat, Clear Chat
12. Chat Details & User Online Status
13. Screenshot Reporting
14. Message Selection

PART B — Rich Media Messages (/api/wie-chat/:chatId/send-*)

15. Send Image Message
16. Send Video Message
17. Send Audio Message
18. Send Document / File Message
19. Send Sticker Message
20. Send Location Message (Static & Live)
21. Send Contact Message
22. Send Profile Message

- 23. Send Event Message
- 24. Get Chat Media & Mark Media Viewed

PART C — Block API (/api/chat-block)

- 25. Block & Unblock User
- 26. Get Blocked Users
- 27. Check Block Status

PART D — Report API (/api/chat-report)

- 28. Report a User
- 29. Get My Reports

PART E — Real-Time Socket Events

- 30. Socket Events Reference
- 31. Full Route Reference
- 32. React Native Integration Examples

PART A — Core Chat API (Base: <http://localhost:5004/api/wie-chat>)

01 Service Overview & Authentication

The WIE Chat Service runs on port 5004 and handles all real-time messaging between users. It provides REST endpoints for message CRUD and uses Socket.IO for live delivery. A gRPC server also runs internally for cross-service communication.

Service Config

```
Base URL (development) : http://localhost:5004
Base URL (Android emu) : http://10.0.2.2:5004
```

Route prefixes:

```
Chat API    → /api/wie-chat
Block API   → /api/chat-block
Report API  → /api/chat-report
```

```
Socket.IO URL: http://10.0.2.2:5004
```

TypeScript

```
// src/services/chatService.ts - Axios instance
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';
import Config from 'react-native-config';

const CHAT_BASE = Config.CHAT_API_URL || 'http://10.0.2.2:5004/api/wie-chat';
const BLOCK_BASE = Config.BLOCK_API_URL || 'http://10.0.2.2:5004/api/chat-block';
const REPORT_BASE = Config.REPORT_API_URL || 'http://10.0.2.2:5004/api/chat-report';

export const chatApi = axios.create({ baseURL: CHAT_BASE, timeout: 20000 });
export const blockApi = axios.create({ baseURL: BLOCK_BASE, timeout: 10000 });
export const reportApi = axios.create({ baseURL: REPORT_BASE, timeout: 10000 });

[chatApi, blockApi, reportApi].forEach(api => {
  api.interceptors.request.use(async cfg => {
    const t = await AsyncStorage.getItem('token');
    if (t) cfg.headers.Authorization = `Bearer ${t}`;
    return cfg;
  });
});
```

02 Chat Permissions Model

Before a chat can be created, the server evaluates chat permissions based on the block status and privacy settings of both users. This is done automatically inside `createOrGetWieChat` — you do not need to call a separate permissions endpoint.

Permission	Meaning
<code>direct</code>	Full chat. Receiver follows sender, or receiver accepts messages from everyone. Messages deliver instantly.
<code>request</code>	Message request. Sender is not followed by receiver. Chat created as <code>type='request'</code> , <code>status='pending'</code> . Receiver must accept before replying.
<code>blocked</code>	Chat cannot be created. Either user has blocked the other, or receiver has DMs fully disabled.

► Permission Decision Flow

Condition	Result
<code>Block exists</code>	→ blocked (no chat possible)
<code>Receiver: allow_from=everyone</code>	→ direct (instant chat)
<code>Receiver: allow_from=followers, sender is follower</code>	→ direct
<code>Receiver: allow_from=followers, sender NOT follower</code>	→ request (if <code>allow_message_requests=true</code>)
<code>Receiver: allow_from=mutuals, both follow each other</code>	→ direct
<code>Receiver: allow_from=mutuals, not mutual</code>	→ request or blocked

Permission results are cached server-side for 5 minutes. Blocking a user immediately invalidates the cache. You never need to call a permissions endpoint — it is evaluated inside `POST /create`.

03 Create or Get Chat

The primary entry point for opening a conversation. If a chat already exists between the two users it is returned; otherwise a new one is created. The server automatically determines chat type (direct or request) based on the follow relationship.

Method	Endpoint	Auth	Description
POST	/create	 Yes	Create or return an existing chat with another user.

► Request Body — application/json

Field	Type	Required	Description
participantId	string	Yes	The user ID of the person to chat with.

► Success Response — Existing Chat (HTTP 200)

JSON Response

```
{
  "success": true,
  "isNew": false,
  "chat": {
    "_id": "chat_abc123",
    "participants": ["user_me", "user_them"],
    "participant": {
      "_id": "user_them",
      "name": "Jane Doe",
      "username": "janedoe",
      "profile_picture": "https://cdn.../avatar.jpg",
      "is_verified": false,
      "isOnline": true,
      "lastSeen": "2025-12-01T11:30:00.000Z"
    },
    "type": "direct",    // 'direct' | 'request'
    "status": "accepted", // 'accepted' | 'pending'
    "messages": [],
    "lastMessage": null,
    "unreadCount": 0,
    "isActive": true,
    "isEmpty": true
  }
}
```

► Success Response — New Chat (HTTP 201)

JSON Response

```
{
  "success": true,
  "isNew": true,
```

```
"isEmpty": true,  
"chat": { /* same shape as above */ }  
}
```

► Error Responses

Error Codes

400	Cannot create chat with yourself
403	Cannot create chat – block exists between users
404	User not found / User is not active

04 List My Chats

Returns all active chats for the authenticated user, sorted by most recent activity. Each item includes the other participant's profile, last message, unread count, and chat type.

Method	Endpoint	Auth	Description
GET	/list	 Yes	Get all chats for the current user, newest first.

► Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Chats per page (default: 40).

► Response

JSON Response

```
{
  "success": true,
  "chats": [
    {
      "_id": "chat_abc123",
      "participant": {
        "_id": "user_them",
        "name": "Jane Doe",
        "username": "janedoe",
        "profile_picture": "https://cdn.../avatar.jpg",
        "is_verified": false,
        "isOnline": false,
        "last_seen_at": "2025-12-01T11:00:00.000Z"
      },
      "lastMessage": {
        "content": "Hey! How are you?",
        "sender": "user_them",
        "timestamp": "2025-12-01T11:00:00.000Z"
      },
      "type": "direct",
      "status": "accepted",
      "unreadCount": 3,
      "updatedAt": "2025-12-01T11:00:00.000Z"
    }
  ]
}
```

Chats with type='request' and status='pending' are message requests from users you don't follow. Show these in a separate 'Requests' tab, not the main chat list.

05 Send Text Message

Sends a text or voice message to a chat. Voice messages are sent by encoding audio as base64 inside the content field with a JSON envelope. After the HTTP response, the server emits Socket.IO events to both participants.

Method	Endpoint	Auth	Description
POST	/send	 Yes	Send a text or voice message. Response arrives before socket events.

► Request Body — application/json

Field	Type	Required	Description
chatId	string	Yes	The _id of the chat to send to.
content	string	Yes	Message text. For voice, send a JSON string with type='voice'.
replyTo	object	No	Optional reply reference: { messageId, sender, content, messageType }.

► Voice Message — content format

TypeScript

```
// Encode audio as base64, wrap in JSON, stringify as content
const voiceContent = JSON.stringify({
  type: 'voice',
  audio: `data:audio/webm;base64,${base64AudioString}`,
  duration: 12.5, // seconds (must be > 0)
  mimeType: 'audio/webm;codecs=opus'
});

await chatApi.post('/send', {
  chatId: 'chat_abc123',
  content: voiceContent
});
```

► Success Response (HTTP 201)

JSON Response

```
{
  "success": true,
  "chatId": "chat_abc123",
  "message": {
    "_id": "msg_001",
    "sender": "user_me",
    "content": "Hello!",
    "messageType": "text",
    "timestamp": "2025-12-01T11:00:00.000Z",
    "readBy": ["user_me"],
```



```
    "deliveredTo": ["user_them"],  
    "isRead": false,  
    "isSender": true,  
    "receiverOnline": false  
  },  
  "chat": { /* updated chat object with participant info */ }  
}
```

06 Get Chat Messages

Returns paginated messages for a chat. Messages are returned newest-last (chronological order). Apply the page parameter to load older messages. Messages deleted for the current user are automatically excluded.

Method	Endpoint	Auth	Description
GET	/:chatId/messages	 Yes	Get paginated messages. Excludes deleted-for-me and cleared messages.

► Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1). Page 2 loads the previous batch of messages.
limit	number	No	Messages per page (default: 50).

► Response

JSON Response

```
{
  "success": true,
  "totalMessages": 142,
  "page": 1,
  "pages": 3,
  "hasMore": true,
  "chat": {
    "_id": "chat_abc123",
    "type": "direct",
    "status": "accepted",
    "participant": {
      "_id": "user_them",
      "name": "Jane Doe",
      "isOnline": true,
      "last_seen_at": "2025-12-01T11:30:00.000Z",
      "lastSeenFormatted": null // null when isOnline=true
    }
  },
  "messages": [
    {
      "_id": "msg_001",
      "sender": "user_me",
      "content": "Hello!",
      "messageType": "text", // text|image|video|audio|file|voice|sticker|
                           // location|live_location|contact|profile|event
      "timestamp": "2025-12-01T11:00:00.000Z",
      "readBy": ["user_me", "user_them"],
      "isRead": true,
      "replyTo": null
    }
  ]
}
```

```
}
```

► Message Types

messageType	Description
text	Plain text message.
image	One or more images. Contains chat_images[] with url, viewMode, viewedBy.
video	Single video. Contains chat_videos[] with url, duration, size, mimeType.
audio	Audio file. Contains chat_audio[] with url, duration, mimeType.
voice	Voice memo. Contains voiceData: { audioBase64, duration, mimeType }.
file	Document/file. Contains chat_files[] with url, name, size, extension.
sticker	GIF sticker. Contains stickerData: { url, stickerId, pack }.
location	Static location. Contains locationData: { latitude, longitude, address, name }.
live_location	Live GPS location. Same as location plus isLive=true, liveExpiry.
contact	Shared contact card. Contains contactData: { name, phone[], email[] }.
profile	Shared WIE profile. Contains profileData: { userId, name, username, avatar }.
event	Shared event. Contains eventData: { eventId, title, venue, startDate }.

07 Chat Suggestions & User Search

Suggestions returns users who follow you (good candidates to start a chat with). Search lets users find anyone by name or username. Both exclude blocked users.

Method	Endpoint	Auth	Description
GET	/suggestions	 Yes	Get chat suggestions (users who follow the caller).
GET	/search?query=	 Yes	Search all users by name or username.

► GET /search — Query Parameters

Field	Type	Required	Description
query	string	Yes	Search term. Must be non-empty.

► Response — both endpoints

JSON Response

```
// /suggestions
{ "success": true, "suggestions": [
  { "_id": "user_abc", "name": "Jane", "username": "jane",
    "profile_picture": "https://...", "is_verified": false }
] }

// /search
{ "success": true, "count": 4, "users": [ /* same shape */ ] }
```

08 Message Requests

When a user who is not followed by the receiver sends the first message, it lands in the Requests inbox instead of the main chat list. The receiver can accept (moves to direct chat) or decline (deletes the chat).

Method	Endpoint	Auth	Description
GET	/requests	✓ Yes	Get all pending message requests for the current user.
POST	/:chatId/accept	✓ Yes	Accept a message request — converts to direct chat.
POST	/:chatId/decline	✓ Yes	Decline and permanently delete a message request.

► GET /requests — Response

JSON Response

```
{
  "success": true,
  "requests": [
    {
      "_id": "chat_req001",
      "participant": { "_id": "user_abc", "name": "...", "username": "..." },
      "lastMessage": { "content": "Hi there!", "sender": "user_abc", "timestamp": "...",
      "type": "request",
      "status": "pending",
      "unreadCount": 1,
      "updatedAt": "2025-12-01T10:00:00.000Z"
    }
  ]
}
```

► POST /:chatId/accept — Response

JSON Response

```
{ "success": true, "message": "Message request accepted" }
```

► POST /:chatId/decline — Response

JSON Response

```
{ "success": true, "message": "Message request declined" }
```

After accepting, the server emits chat-list-update to both participants via Socket.IO with type='direct' and status='accepted'. Listen for this event to update the UI without a full re-fetch.

09 Mark Read & Unread

Method	Endpoint	Auth	Description
POST	/:chatId/mark-read	✓ Yes	Mark all unread messages in a chat as read. Emits messages-read socket event to sender.
POST	/:chatId/mark-unread	✓ Yes	Mark the last message from the other user as unread.

► POST /:chatId/mark-read — Response

JSON Response

```
{
  "success": true,
  "message": "Messages marked as read",
  "markedCount": 5
}
```

Always call mark-read when the user opens a chat screen. This triggers the 'messages-read' Socket.IO event that shows double ticks (✓✓) for the sender.

10 Unread Counts

Two endpoints return unread counts: one gives the total count of chats with unread messages (for the notification badge on the Messages tab icon), the other gives a finer count by user.

Method	Endpoint	Auth	Description
GET	/unread-count	 Yes	Number of chats with at least one unread message (for tab badge).
GET	/unread-users-count	 Yes	Number of unique users with unread messages.

► GET /unread-count — Response

JSON Response

```
{ "success": true, "unreadCount": 4 }
```

► GET /unread-users-count — Response

JSON Response

```
{ "success": true, "unreadUsersCount": 4 }
```

11 Delete Message, Delete Chat, Clear Chat

Method	Endpoint	Auth	Description
DELETE	<code>/:chatId/messages/:messageId</code>	✓ Yes	Hard-delete a single message (sender only). Removes from DB.
POST	<code>/:chatId/messages/delete-for-me</code>	✓ Yes	Soft-delete selected messages for current user only.
POST	<code>/:chatId/messages/delete-for-everyone</code>	✓ Yes	Delete selected messages for all participants (sender only).
DELETE	<code>/:chatId/clear</code>	✓ Yes	Clear all messages in a chat for current user only (timestamp-based).
DELETE	<code>/:chatId</code>	✓ Yes	Delete entire chat for current user. If both delete, chat is removed.
DELETE	<code>/:chatId/delete-for-me</code>	✓ Yes	Mark all messages as deleted for current user, add to deletedFor.

► POST `/:chatId/messages/delete-for-me` — Body

Field	Type	Required	Description
<code>messageIds</code>	<code>string[]</code>	Yes	Array of message <code>_id</code> strings to delete for the current user only.

► POST `/:chatId/messages/delete-for-everyone` — Body

Field	Type	Required	Description
<code>messageIds</code>	<code>string[]</code>	Yes	Array of message <code>_id</code> strings. All must be sent by the current user.

► DELETE `/:chatId/messages/:messageId` — Response

JSON Response

```
{
  "success": true,
  "message": "Message deleted successfully",
  "lastMessage": { /* updated last message or null */ }
}
```


12 Chat Details & User Online Status

Method	Endpoint	Auth	Description
GET	/:chatId/get-chat-details	✓ Yes	Full chat document including isBlocked and isBlockedBy fields.
GET	/user/:userId/status	✓ Yes	Check if a user is online and when they were last seen.

► GET /:chatId/get-chat-details — Response

JSON Response

```
{
  "success": true,
  "chat": {
    "_id": "chat_abc",
    "participants": ["user_me", "user_them"],
    "type": "direct",
    "status": "accepted",
    "isBlocked": false,
    "isBlockedBy": undefined // 'you' | 'them' | undefined
  }
}
```

► GET /user/:userId/status — Response

JSON Response

```
{ "success": true, "isOnline": true, "lastSeen": "2025-12-01T11:30:00.000Z" }
```

13 Screenshot Reporting

When a user takes a screenshot of a conversation, the client reports it to the server. The server deduplicates via Redis (48-hour window per viewer per chat) and emits a real-time alert to the chat owner via Socket.IO.

Method	Endpoint	Auth	Description
POST	/:chatId/report-screenshot	 Yes	Report a screenshot event. Notifies the other participant once per 48h.

► Request Body

Field	Type	Required	Description
platform	string	No	Client platform: 'ios' 'android' 'web'. Default: 'web'.
imageBase64	string	No	Optional base64 screenshot image for ML verification.

► Response

JSON Response

```
{
  "success": true,
  "alerted": true,
  "count": 3,
  "confidence": 1.0,
  "title": "📷 janedoe took a screenshot",
  "message": "janedoe screenshotted your conversation"
}
```

The socket event 'chat-screenshot-alert' is emitted to the chat owner in real time. The 'chat-screenshot-event' with isSelfConfirmation=true is emitted back to the screenshotter.

14 Message Selection

Returns a minimal list of visible messages for the selection mode UI (long-press to select multiple messages). Only includes non-deleted messages and minimal fields for performance.

Method	Endpoint	Auth	Description
GET	/:chatId/messages/selection	 Yes	Get minimal message list for bulk selection UI.

► Response

JSON Response

```
{
  "success": true,
  "messages": [
    { "_id": "msg_001", "sender": "user_me",
      "content": "Hello!", "timestamp": "...", "isSender": true }
  ]
}
```

PART B — Rich Media Messages (/api/wie-chat/:chatId/send-*)

15 Send Image Message

Upload one or more images to a chat. Accepts multipart/form-data with up to 10 image files. Supports viewMode for view-once and allow-replay behaviors.

Method	Endpoint	Auth	Description
POST	/:chatId/send-image	 Yes	Send 1–10 images. Multipart/form-data. Files uploaded to Cloudinary.

► Form Fields

Field	Type	Required	Description
images	File[]	Yes	Up to 10 image files (jpg/png/gif/webp). Field name: 'images'.
caption	string	No	Optional caption shown below the image.
viewMode	string	No	'keep' (default) 'view_once' 'allow_replay'

► React Native — Send Image

TypeScript

```
const sendImage = async (chatId: string, assets: ImagePickerAsset[]) => {
  const formData = new FormData();
  assets.forEach(asset => {
    formData.append('images', {
      uri: asset.uri,
      name: asset.fileName || 'photo.jpg',
      type: asset.type || 'image/jpeg',
    } as any);
  });
  formData.append('caption', 'Check this out!');
  formData.append('viewMode', 'keep');

  return chatApi.post(`/${chatId}/send-image`, formData, {
    headers: { 'Content-Type': 'multipart/form-data' },
  });
};
```

► Response — message.chat_images[]

JSON Response

```
{
  "success": true,
  "message": {
    "messageType": "image",
```

```
"chat_images": [
  { "url": "https://res.cloudinary.com/.../photo.jpg",
    "viewMode": "keep",
    "viewedBy": [] }
]
```

viewMode	Behaviour
keep	Image stays visible indefinitely. No auto-delete.
view_once	Image disappears after the recipient views it once.
allow_replay	Image can be viewed multiple times but is tracked.

16 Send Video Message

Method	Endpoint	Auth	Description
POST	/:chatId/send-video	 Yes	Send a single video. multipart/form-data. Field name: 'video'.

► Form Fields

Field	Type	Required	Description
video	File	Yes	Single video file (mp4/mov/webm). Field name: 'video'.
caption	string	No	Optional caption.
viewMode	string	No	'keep' 'view_once' 'allow_replay'. Default: 'keep'.

► Response — message.chat_videos[]

JSON Response

```
{ "chat_videos": [  
  { "url": "https://...", "duration": 12.4, "size": 2048000,  
    "mimeType": "video/mp4", "viewMode": "keep", "viewedBy": [] }  
]
```

17 Send Audio Message

Method	Endpoint	Auth	Description
POST	/:chatId/send-audio	 Yes	Send an audio file. multipart/form-data. Field name: 'audio'.

► Form Fields

Field	Type	Required	Description
audio	File	Yes	Audio file (mp3/wav/ogg/m4a/webm). Field name: 'audio'.

► Response — message.chat_audio[]

JSON Response

```
{ "chat_audio": [  
  { "url": "https://...", "duration": 8.2, "mimeType": "audio/mp3",  
    "originalName": "recording.mp3" }  
]
```

18 Send Document / File Message

Method	Endpoint	Auth	Description
POST	/:chatId/send-document	 Yes	Send a document or file. multipart/form-data. Field name: 'document'.

► Form Fields

Field	Type	Required	Description
document	File	Yes	Any document (pdf/doc/xlsx/zip/etc). Field name: 'document'.

► Response — message.chat_files[]

JSON Response

```
{ "chat_files": [  
  { "url": "https://...", "name": "report.pdf",  
    "size": 512000, "mimeType": "application/pdf", "extension": "pdf" }  
]
```


19 Send Sticker Message

Sends a Giphy GIF sticker. Get sticker URLs from the `/api/flux/stickers/trending` or `/api/flux/stickers/search` endpoints on the Media Service.

Method	Endpoint	Auth	Description
POST	<code>/:chatId/send-sticker</code>	 Yes	Send a Giphy sticker. Body: JSON. No file upload.

► Request Body — application/json

Field	Type	Required	Description
<code>url</code>	string	Yes	Full Giphy GIF URL.
<code>stickerId</code>	string	No	Giphy sticker ID for reference.
<code>pack</code>	string	No	Pack or category name.

► Response — message.stickerData

JSON Response

```
{ "stickerData": { "url": "https://media.giphy.com/.../200.gif",  
  "stickerId": "abc", "pack": null } }
```

20 Send Location Message (Static & Live)

Two location modes: static (a pin on a map) and live (real-time GPS coordinates that update until the expiry time). Live location can be updated continuously via PATCH.

Method	Endpoint	Auth	Description
POST	/:chatId/send-location	✓ Yes	Send a static or live location pin.
PATCH	/:chatId/live-location/:messageId	✓ Yes	Update coordinates of an active live-location message.

► POST /:chatId/send-location — Body

Field	Type	Required	Description
latitude	number	Yes	Decimal degrees latitude.
longitude	number	Yes	Decimal degrees longitude.
address	string	No	Human-readable address string.
name	string	No	Place name or label.
isLive	boolean	No	Set to true to create a live location message.
liveExpiry	string	No	ISO 8601 datetime when live location stops. Required if isLive=true.

► PATCH /:chatId/live-location/:messageId — Body

Field	Type	Required	Description
latitude	number	Yes	Updated latitude.
longitude	number	Yes	Updated longitude.

► Response — message.locationData

JSON Response

```
{ "locationData": {  
  "latitude": 51.5074,  
  "longitude": -0.1278,  
  "address": "London, UK",  
  "name": "Big Ben",  
  "isLive": false,  
  "liveExpiry": null  
} }
```

21 Send Contact Message

Method	Endpoint	Auth	Description
POST	/:chatId/send-contact	 Yes	Share a contact card. Body: JSON.

► Request Body — application/json

Field	Type	Required	Description
name	string	Yes	Contact's full name.
phone	string string[]	Yes	One or more phone numbers.
email	string string[]	No	One or more email addresses.
avatar	string	No	Profile image URL of the contact.
vCard	string	No	Full vCard string for import.

► Response — message.contactData

JSON Response

```
{ "contactData": { "name": "Alice", "phone": ["+44 7700 000000"],  
  "email": ["alice@example.com"], "avatar": null, "vCard": null } }
```

22 Send Profile Message

Shares another WIE user's profile card in the chat. Resolve the user details from the User Service before calling this endpoint, then pass them in the body.

Method	Endpoint	Auth	Description
POST	/:chatId/send-profile	 Yes	Share a WIE user profile card. Body: JSON.

► Request Body — application/json

Field	Type	Required	Description
profileUserId	string	Yes	User ID of the profile being shared.
name	string	No	Display name (server fetches if missing).
username	string	No	Username (server fetches if missing).
avatar	string	No	Profile picture URL.
bio	string	No	Bio text.
is_verified	boolean	No	Verified badge status.

23 Send Event Message

Method	Endpoint	Auth	Description
POST	/:chatId/send-event	 Yes	Share an event card. Body: JSON.

► Request Body — application/json

Field	Type	Required	Description
eventId	string	Yes	Event ID from the event/ticketing system.
title	string	Yes	Event title.
description	string	No	Short event description.
startDate	string	No	ISO 8601 start datetime.
endDate	string	No	ISO 8601 end datetime.
venue	string	No	Venue name or address.
image	string	No	Event banner image URL.
ticketUrl	string	No	URL to purchase tickets.

24 Get Chat Media & Mark Media Viewed

Method	Endpoint	Auth	Description
GET	/:chatId/media	✓ Yes	Get paginated media messages for a chat (images, videos, audio, files).
POST	/:chatId/messages/:messageId/viewed	✓ Yes	Mark view_once or allow_replay media as viewed by current user.

► GET /:chatId/media — Query Parameters

Field	Type	Required	Description
type	string	No	Filter by: image video audio file sticker voice. Omit for all media.
page	number	No	Page number (default: 1).
limit	number	No	Items per page (default: 20).

► GET /:chatId/media — Response

JSON Response

```
{ "success": true, "total": 48, "page": 1, "pages": 3, "hasMore": true,
  "media": [ /* message objects with rich fields */ ] }
```

PART C — Block API (Base: http://localhost:5004/api/chat-block)

25 Block & Unblock User

Blocking a user removes all follow relationships (both directions via gRPC) and prevents any new messages between them. Existing chats remain visible but no new messages can be sent. Unblocking restores the chat visibility.

Method	Endpoint	Auth	Description
POST	/block	✓ Yes	Block a user. Removes follow relationships. Emits socket events.
DELETE	/unblock/:userId	✓ Yes	Unblock a previously blocked user.

► POST /block — Request Body

Field	Type	Required	Description
userId	string	Yes	The user ID of the person to block.

► POST /block — Response (HTTP 200)

JSON Response

```
{ "success": true, "message": "User blocked successfully" }
```

► DELETE /unblock/:userId — Response (HTTP 200)

JSON Response

```
{ "success": true, "message": "User unblocked successfully" }
```

► Socket Events Emitted on Block

Socket Event	Description
user-blocked-you	Sent to the BLOCKED user. Contains { blockerId, chatIds }.
you-blocked-user	Sent to the BLOCKER. Contains { blockedId, chatIds }.
user-unblocked-you	Sent to the UNBLOCKED user when block is removed.
you-unblocked-user	Sent to the UNBLOCKER when block is removed.

26 Get Blocked Users

Method	Endpoint	Auth	Description
GET	/blocked	 Yes	Get the current user's blocked users list (paginated).

► Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Results per page (default: 50).

► Response

JSON Response

```
{
  "success": true,
  "total": 2,
  "page": 1,
  "totalPages": 1,
  "blockedUsers": [
    { "userId": "user_abc", "blockedAt": "2025-11-01T10:00:00.000Z" }
  ]
}
```


27 Check Block Status

Returns whether the current user has blocked the target and whether the target has blocked the current user. Both directions are checked in a single call.

Method	Endpoint	Auth	Description
GET	/check-block-status/:userId	 Yes	Check bidirectional block status between caller and target.

► Response

JSON Response

```
{
  "success": true,
  "iBlockedThem": false,
  "theyBlockedMe": true
}
```

PART D — Report API (Base: <http://localhost:5004/api/chat-report>)

28 Report a User

Submits a report about another user's behaviour. Duplicate reports for the same user within 24 hours are rejected. Reports are reviewed by the WIE moderation team and do not auto-block or notify the reported user.

Method	Endpoint	Auth	Description
POST	/report	 Yes	Submit a user report. One report per user per 24 hours.

► Request Body — application/json

Field	Type	Required	Description
userId	string	Yes	User ID of the person being reported.
reportType	string	Yes	One of: harassment spam inappropriate threat other
reason	string	Yes	Detailed reason text explaining the report.
chatId	string	No	Chat ID for context (helps moderation team review messages).
messageIds	string[]	No	Specific message IDs as evidence.

► Report Types

reportType	Description
harassment	Bullying, threats, or repeated unwanted contact.
spam	Unsolicited promotional or repetitive messages.
inappropriate	Offensive content (hate speech, graphic material, etc.).
threat	Direct threats of harm or violence.
other	Anything that doesn't fit the above categories.

► Success Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "message": "Report submitted successfully. Our team will review it.",
  "reportId": "report_abc123"
}
```

► Error Responses

Error Codes

```
400 You have already reported this user recently - 24h cooldown active
400 Invalid report type - must be one of the five valid values
```

29 Get My Reports

Method	Endpoint	Auth	Description
GET	/my-reports	 Yes	Get reports submitted by the current user (paginated).

► Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Results per page (default: 20).

► Response

JSON Response

```
{
  "success": true,
  "total": 1,
  "page": 1,
  "totalPages": 1,
  "reports": [
    {
      "reportedId": "user_abc",
      "reportType": "spam",
      "reason": "Sending repeated promotional links",
      "status": "pending", // 'pending' | 'reviewed' | 'resolved'
      "createdAt": "2025-12-01T10:00:00.000Z"
    }
  ]
}
```

PART E — Real-Time Socket Events (Socket.IO: <http://localhost:5004>)

30 Socket Events Reference

Connect to the Socket.IO server and join your personal room (userId) to receive notifications. When inside a chat, also join the chatId room for message events.

TypeScript

```
// React Native — Socket.IO setup
import { io } from 'socket.io-client';
import AsyncStorage from '@react-native-async-storage/async-storage';

const socket = io('http://10.0.2.2:5004', {
  transports: ['websocket'],
  autoConnect: false,
});

// Connect with token
const connectSocket = async () => {
  const token = await AsyncStorage.getItem('token');
  socket.auth = { token };
  socket.connect();
  socket.emit('join-user-room', { userId: myUserId });
};

// Join a chat room when opening a chat screen
const joinChat = (chatId: string) =>
  socket.emit('join-chat-room', { chatId });

// Leave a chat room when closing a chat screen
const leaveChat = (chatId: string) =>
  socket.emit('leave-chat-room', { chatId });
```

► Inbound Socket Events — Chat & Messages

Event Name	Payload Summary
<code>new-message</code>	New message delivered to a chat room. { chatId, message, sender, timestamp }
<code>new-message-notification</code>	Notification for new message — update chat list. { chatId, message, participant, type, status, unreadCount }
<code>new-message-request</code>	New message request from a non-follower. { chatId, message, isNewRequest: true }
<code>messages-read</code>	Recipient read your messages — show double tick. { chatId, userId, messageIds }
<code>chat-unread-update</code>	Update unread badge for a specific chat. { chatId, unreadCount }

Event Name	Payload Summary
<code>chat-list-update</code>	Re-order or update the chat list. { chatId, lastMessage, participant, type, status, unreadCount }
<code>message-deleted</code>	A message was deleted for everyone. { chatId, messageIds, lastMessage }
<code>messages-deleted-for-me</code>	Messages deleted for current user only. { chatId, messageIds, lastMessage }
<code>messages-deleted-for-everyone</code>	Messages deleted for all. { chatId, messageIds, lastMessage }
<code>chat-cleared</code>	Chat cleared by a user. { chatId, clearedBy }
<code>chat-deleted</code>	Chat deleted. { chatId, deletedBy, permanent }
<code>chat-deleted-for-me</code>	Chat hidden for current user. { chatId, deletedBy }

► Inbound Socket Events — Requests & Unread

Event Name	Payload Summary
<code>request-messages-read</code>	Request chat was read. { chatId, timestamp }
<code>request-count-update</code>	Update request badge count. { chatId, unreadCount }

► Inbound Socket Events — Block & Screenshot

Event Name	Payload Summary
<code>user-blocked-you</code>	Someone blocked the current user. { blockerId, chatIds }
<code>you-blocked-user</code>	Confirmation you blocked someone. { blockedId, chatIds }
<code>user-unblocked-you</code>	Someone unblocked the current user.
<code>you-unblocked-user</code>	Confirmation you unblocked someone.
<code>chat-screenshot-alert</code>	Another user took a screenshot of your chat. { chatId, viewerName, totalCount, title }
<code>chat-screenshot-event</code>	Screenshot event for display in chat. isSelfConfirmation=true sent back to screenshotter.
<code>live-location-update</code>	Updated coordinates for a live-location message. { chatId, messageId, latitude, longitude }
<code>media_viewed</code>	Another user viewed a view_once/allow_replay media. { chatId, messageId, viewedBy }

31 Full Route Reference

► Chat API (/api/wie-chat)

Method	Endpoint	Auth	Description
GET	/suggestions	✓ Yes	Chat suggestions (followers of current user)
GET	/search?query=	✓ Yes	Search users for chat
POST	/create	✓ Yes	Create or return existing chat
GET	/list	✓ Yes	All chats (paginated)
GET	/requests	✓ Yes	Message requests inbox
GET	/unread-count	✓ Yes	Chats with unread messages count
GET	/unread-users-count	✓ Yes	Unique users with unread messages
POST	/send	✓ Yes	Send text or voice message
GET	/user/:userId/status	✓ Yes	User online status
GET	/:chatId/get-chat-details	✓ Yes	Full chat document + block status
POST	/:chatId/mark-read	✓ Yes	Mark all messages as read
POST	/:chatId/mark-unread	✓ Yes	Mark last message as unread
GET	/:chatId/messages	✓ Yes	Paginated chat messages
POST	/:chatId/accept	✓ Yes	Accept message request
POST	/:chatId/decline	✓ Yes	Decline message request
DELETE	/:chatId/clear	✓ Yes	Clear chat for current user
DELETE	/:chatId/messages/:messageId	✓ Yes	Hard-delete a message (sender only)
POST	/:chatId/messages/delete-for-me	✓ Yes	Delete selected messages for me only
POST	/:chatId/messages/delete-for-everyone	✓ Yes	Delete selected messages for all
POST	/:chatId/report-screenshot	✓ Yes	Report screenshot event
DELETE	/:chatId	✓ Yes	Delete chat for current user
GET	/:chatId/messages/selection	✓ Yes	Minimal message list for selection UI
DELETE	/:chatId/delete-for-me	✓ Yes	Mark all messages deleted for me

Method	Endpoint	Auth	Description
POST	/:chatId/send-image	✓ Yes	Send image(s) — multipart
POST	/:chatId/send-video	✓ Yes	Send video — multipart
POST	/:chatId/send-audio	✓ Yes	Send audio file — multipart
POST	/:chatId/send-document	✓ Yes	Send document — multipart
POST	/:chatId/send-sticker	✓ Yes	Send GIF sticker — JSON
POST	/:chatId/send-location	✓ Yes	Send static or live location — JSON
PATCH	/:chatId/live-location/:messageId	✓ Yes	Update live location coordinates
POST	/:chatId/send-contact	✓ Yes	Send contact card — JSON
POST	/:chatId/send-profile	✓ Yes	Send user profile card — JSON
POST	/:chatId/send-event	✓ Yes	Send event card — JSON
POST	/:chatId/messages/:messageId/viewed	✓ Yes	Mark view_once media as viewed
GET	/:chatId/media	✓ Yes	Get media messages for a chat

► Block API (/api/chat-block)

Method	Endpoint	Auth	Description
POST	/block	✓ Yes	Block a user (removes follow relationships)
DELETE	/unblock/:userId	✓ Yes	Unblock a user
GET	/blocked	✓ Yes	Get blocked users list
GET	/check-block-status/:userId	✓ Yes	Check bidirectional block status

► Report API (/api/chat-report)

Method	Endpoint	Auth	Description
POST	/report	✓ Yes	Submit a user report
GET	/my-reports	✓ Yes	Get my submitted reports

32 React Native Integration Examples

► Complete chatService.ts (Core Functions)

TypeScript

```
// src/services/chatService.ts
import { chatApi, blockApi, reportApi } from './apiClients';

// — Core Chat —
export const createOrGetChat = (participantId: string) =>
  chatApi.post('/create', { participantId }).then(r => r.data);

export const getMyChats = (page = 1) =>
  chatApi.get('/list', { params: { page } }).then(r => r.data);

export const getChatMessages = (chatId: string, page = 1) =>
  chatApi.get(`/${chatId}/messages`, { params: { page } }).then(r => r.data);

export const sendMessage = (chatId: string, content: string, replyTo?: any) =>
  chatApi.post('/send', { chatId, content, replyTo }).then(r => r.data);

export const markAsRead = (chatId: string) =>
  chatApi.post(`/${chatId}/mark-read`).then(r => r.data);

export const getUnreadCount = () =>
  chatApi.get('/unread-count').then(r => r.data.unreadCount);

export const getRequests = () =>
  chatApi.get('/requests').then(r => r.data.requests);

export const acceptRequest = (chatId: string) =>
  chatApi.post(`/${chatId}/accept`).then(r => r.data);

export const declineRequest = (chatId: string) =>
  chatApi.post(`/${chatId}/decline`).then(r => r.data);

// — Block —
export const blockUser = (userId: string) =>
  blockApi.post('/block', { userId }).then(r => r.data);

export const unblockUser = (userId: string) =>
  blockApi.delete(`/unblock/${userId}`).then(r => r.data);

export const checkBlockStatus = (userId: string) =>
  blockApi.get(`/check-block-status/${userId}`).then(r => r.data);

// — Report —
export const reportUser = (body: {
  userId: string; reportType: string; reason: string;
  chatId?: string; messageIds?: string[];
}) => reportApi.post('/report', body).then(r => r.data);
```

► Chat Screen — Open & Listen

TypeScript

```
// When user opens a chat
useEffect(() => {
  socket.emit('join-chat-room', { chatId });
  fetchMessages();
  markAsRead(chatId);

  // New incoming message
  socket.on('new-message', ({ message }) => {
    setMessages(prev => [...prev, message]);
    markAsRead(chatId); // auto-read if chat is open
  });

  // Other user read our messages - show double tick
  socket.on('messages-read', ({ messageIds }) => {
    setMessages(prev => prev.map(m =>
      messageIds.includes(m._id) ? { ...m, isRead: true } : m
    ));
  });

  return () => {
    socket.emit('leave-chat-room', { chatId });
    socket.off('new-message');
    socket.off('messages-read');
  };
}, [chatId]);
```